



Capítulo 3 – Personaliza tu Copilot

Skills · Agentes · MCP · Memory

Marzo 2026

¿Dónde estamos?

Tres sesiones, un objetivo: que Copilot trabaje como TÚ quieres



Cap 1

Entendimos la máquina

LLMs, prompting, modelos,
ventana de contexto



Cap 2

Aprendimos a conducir

Ask→Plan→Agent, entornos,
instructions, referencias



Cap 3

Personaliza tu Copilot

Skills, agentes, MCP,
memory, workflows

"En las dos sesiones anteriores aprendisteis a usar Copilot. Hoy vamos a personalizar Copilot para que trabaje COMO VOSOTROS queréis."

Lo que vamos a construir hoy

Iremos archivo por archivo. Al final, tendréis un `.github/` completo para vuestros repos.

<code>.github/</code>	
├─ <code>copilot-instructions.md</code>	✓ Ya lo tenéis (sesión 2)
├─ <code>copilotignore</code>	NEW Seguridad y contexto limpio
├─ <code>instructions/</code>	
│ └─ <code>readme.instructions.md</code>	✓ Ya lo tenéis (sesión 2)
│ └─ <code>dotnet.instructions.md</code>	NEW Reglas .NET
│ └─ <code>cobol.instructions.md</code>	NEW Reglas COBOL
│ └─ <code>tests.instructions.md</code>	NEW Reglas de testing
├─ <code>prompts/</code>	
│ └─ <code>analyze-jira.prompt.md</code>	NEW Analizar tareas
│ └─ <code>generate-tests.prompt.md</code>	NEW Generar tests
│ └─ <code>review-code.prompt.md</code>	NEW Code review
│ └─ <code>debug-from-logs.prompt.md</code>	NEW Debug desde Elastic
│ └─ <code>document-module.prompt.md</code>	NEW Documentación
├─ <code>agents/</code>	
│ └─ <code>analyst.md</code>	NEW Análisis funcional
│ └─ <code>reviewer.md</code>	NEW Code review
│ └─ <code>cobol-expert.md</code>	NEW Experto COBOL
├─ <code>skills/</code>	
│ └─ <code>dotnet-review/SKILL.md</code>	NEW SKILLS (lo nuevo!)
│ └─ <code>cobol-analyzer/SKILL.md</code>	NEW Review .NET
└─ <code>.vscode/mcp.json</code>	NEW Analizador COBOL
<code>.vs/mcp.json</code>	NEW MCP (VS Code)
<code>AGENTS.md</code>	NEW MCP (Visual Studio 2022)
	NEW Compat. cruzada

19 archivos · 100% replicable · Al final os lleváis la plantilla completa en ZIP

01

Skills de Copilot

Las capacidades del agente — y cómo crear las tuyas

Tools vs Skills — aclarando la confusión

Son cosas diferentes. Entenderlo es clave para personalizar Copilot.

Tools (herramientas built-in)

Capacidades que YA tiene el agente:

`codebase` — busca en el proyecto

`editFiles` — crea/modifica archivos

`terminal` — ejecuta comandos

`fetch` — lee URLs

`githubRepo` — issues, PRs

No los creas tú. Ya existen.

Skills (paquetes de conocimiento)

Conocimiento especializado que TÚ creas:

`SKILL.md` + instrucciones

`references/` — docs adicionales

`sample_codes/` — ejemplos

`scripts/` — automatización

*Tú los creas. Copilot los descubre
y los usa automáticamente.*



Analogía: los tools son los **brazos y ojos** del agente (capacidades). Los skills son **libros de recetas** que le enseñan CÓMO hacer tareas específicas.

Los skills son un estándar abierto: funcionan en Copilot, Claude Code, Cursor y más.

La jerarquía completa de personalización

5 niveles, cada uno con un propósito diferente

Nivel	Archivo	Invocación	Se carga...	Ideal para
Instructions	copilot-instructions.md *.instructions.md	Automática (siempre)	En cada request	Convenciones, stack, reglas globales
Prompt files	*.prompt.md en .github/prompts/	Manual /nombre	Solo al invocar	Tareas simples repetitivas
Skills ★	SKILL.md + recursos en .github/skills/	/nombre O auto-descubierto	Progressive: meta → body → recursos	Tareas complejas con recursos
Agents	*.agent.md en .github/agents/	Manual @nombre	Mientras lo uses en la sesión	Roles persistentes orquestración
MCP	mcp.json (.vscode/ o .vs/)	Automática (agente decide)	Cuando el agente necesita la herramienta	Conectar con servicios externos

Skills reemplazan al patrón de prompt files según el repo oficial awesome-copilot. Ventajas: auto-descubrimiento, progressive loading, recursos adjuntos, estándar abierto.

Pero los prompt files siguen siendo útiles para tareas simples sin recursos.

⚠ Skills funcionan en VS Code + CLI + Coding Agent. En Visual Studio 2022 aún NO están soportados.

⚡ Anatomía de un Skill

Carpeta con SKILL.md + recursos opcionales en `.github/skills/`

```
.github/skills/dotnet-review/  
├── SKILL.md          ← Obligatorio  
└── references/  
    └── checklist.md ← Recurso (carga bajo demanda)
```

```
--- SKILL.md ---  
---
```

```
name: dotnet-review
```

```
description: >
```

```
  Revisa código .NET contra las convenciones  
  del equipo. Usa cuando te pidan hacer code  
  review o validar calidad de código C#.
```

```
---
```

```
# Code Review .NET
```

1. Lee el archivo a revisar.
2. Ejecuta dotnet build.
3. Aplica [checklist](references/checklist.md).
4. Genera informe con hallazgos.

```
Prioridad:
```

```
Seguridad > Correctness > Rendimiento
```

📁 `.github/skills/dotnet-review/SKILL.md`

Progressive loading

1. Metadata (~100 palabras)

name + description siempre visible

2. Body (si es relevante)

Instrucciones del SKILL.md

3. Recursos (bajo demanda)

references/, scripts/, etc.

Invocación

`/dotnet-review`

Manual desde el chat

O automático:

Copilot lee la description
y lo activa si es relevante

Dónde se guardan

Proyecto: `.github/skills/`

Personal: `~/.copilot/skills/`

Recursos: skills que ya existen

No empezáis de cero — hay skills oficiales y comunitarios listos para usar

dotnet/skills (Microsoft oficial)

Skills .NET del equipo que hace la plataforma. Instalables como plugins.

github.com/dotnet/skills

github/awesome-copilot

Colección comunitaria: skills, agents, instructions, prompts. Curada por GitHub.

github.com/github/awesome-copilot

Microsoft Skill Creator — un skill que crea otros skills

Disponible en awesome-copilot. Usa el MCP de Microsoft Learn para investigar documentación oficial y generar skills estructurados automáticamente.

Cómo funciona:

1. Le pides: "Crea un skill para Azure Functions en .NET 8"
2. Investiga en Microsoft Learn (docs, code samples, best practices)
3. Genera un SKILL.md + references/ + sample_codes/ completo
4. Tú revisas, ajustas y lo guardas en .github/skills/

Incluye 4 templates: SDK/Library, Azure Service, Framework, API/Protocol

Estos recursos se instalan copiando la carpeta del skill a .github/skills/ de tu repo.

Ejercicio 0 — Crea tu primer skill

.NET (VS Code o CLI)

1. Crea la carpeta:
`.github/skills/dotnet-review/`
2. Crea SKILL.md con:
`name, description, instrucciones de review`
3. Añade references/checklist.md
4. Prueba: `/dotnet-review` en el chat
5. Verifica: `/skills list (CLI)`

COBOL (VS Code o CLI)

1. Crea la carpeta:
`.github/skills/cobol-analyzer/`
2. Crea SKILL.md con:
`name, description, análisis de PERFORMs`
3. Añade references/common-patterns.md
4. Prueba: `/cobol-analyzer` en el chat
5. Verifica: `/skills list (CLI)`

 **Visual Studio 2022:** Skills aún no soportados. Cread el skill igualmente (es un archivo .md) y usadlo desde VS Code o CLI. *El skill estará listo para cuando VS 2022 lo soporte.*

 10 minutos · Usa la plantilla ZIP como referencia · Comparte qué skill has creado

02

Viajando por `.github/`

Archivo por archivo, construimos tu configuración personalizada



Lo que Copilot NO debe ver — seguridad y contexto limpio

```
# Secrets y configuración sensible
appsettings.Production.json
appsettings.Staging.json
**/secrets/
*.pfx *.key *.pem

# Datos de producción
**/testdata/production-*.json

# Dependencias (ensucian la ventana de contexto)
**/node_modules/
**/bin/
**/obj/
**/packages/

# Archivos generados
**/Migrations/*.Designer.cs
**/*.generated.cs
```

Seguridad

No enviar secrets, connection strings ni tokens a los servidores de GitHub/OpenAI/Anthropic

Contexto limpio

No desperdiciar tokens con bin/, obj/ y código de terceros. Más espacio para TU código.

 `.github/copilotignore` — Funciona exactamente como `.gitignore`

Instructions por lenguaje

Se activan SOLO cuando editas archivos que coinciden con applyTo

```
---
applyTo: "**/*.cs,**/*.csproj"
---
# Instrucciones .NET
## Versión y framework
- C# 12, .NET 8. No sugerir APIs de .NET Framework.
- Entity Framework Core 8 para acceso a datos.

## Patrones obligatorios
- Inyección de dependencias siempre.
- async/await en toda operación I/O.
- Result pattern para errores de dominio.
- DTOs para entradas/salidas de API.

## Testing
- xUnit + FluentAssertions + NSubstitute
- Arrange-Act-Assert
- Nombre: [Metodo]_[Escenario]_[Resultado]
```

```
---
applyTo: "**/*.cob,/**/*.cpy,**/*.jcl"
---
# Instrucciones COBOL
## Formato estricto
- Columnas 7-72. NUNCA fuera de rango.
- Comentarios: * en columna 7.
- Continuación: - en columna 7.

## Estructura
- Incluir copybooks con COPY ... REPLACING.
- PERFORM THRU con párrafos delimitados.
- 88-levels para condiciones booleanas.

## Precauciones
- No modificar copybooks sin listar
  qué otros programas los usan.
- No inventar DD names.
```

 .github/instructions/dotnet.instructions.md

 .github/instructions/cobol.instructions.md

También hay tests.instructions.md con applyTo: "**/*Tests.cs" — reglas específicas para archivos de test

¿Cuándo usar cuál?

Tipo	¿Cuándo?	Ejemplo
<code>copilot-instructions.md</code>	Reglas para TODO el repo	"Respondemos en español" "Clean Architecture"
<code>*.instructions.md</code> con <code>applyTo</code>	Reglas de un lenguaje o área específica	"COBOL: columnas 7-72" ".NET: async siempre"
Personal (IDE settings)	Preferencias individuales (máxima prioridad)	"Respuestas concisas" "Prefiero ejemplos"



Regla práctica: Si aplica a todos → `copilot-instructions.md`. Si solo a un tipo de archivo → `*.instructions.md`. Si es personal → settings del IDE.

⚡ Prompt Files avanzados

De prompts manuales a tareas estandarizadas con /nombre

¿Cuántas veces habéis escrito algo así?

"Explicame qué hace este código paso a paso"

"Genera tests unitarios para esta función"

"Revisa este código y busca problemas"

"Documenta estos métodos públicos"



/analyze-jira

Analizar tarea de Jira

/generate-tests

Generar tests unitarios

/review-code

Code review con checklist

/debug-from-logs

Debug desde Elastic

/document-module

Documentar un módulo

Variables disponibles en prompt files

Variable	Contenido	Variable	Contenido
<code>\${file}</code>	Ruta del archivo activo	<code>\${selection}</code>	Código seleccionado
<code>\${filename}</code>	Nombre del archivo	<code>\${workspaceFolder}</code>	Raíz del proyecto
<code>\${filenameNoExt}</code>	Nombre sin extensión	<code>\${input:n:hint}</code>	Pide input al usuario!

Ejemplo: /analyze-jira

📁 .github/prompts/analyze-jira.prompt.md

```
---
description: "Analiza una tarea de Jira y propone plan"
agent: "agent"
model: "claude-sonnet-4"
tools: ["codebase", "read"]
---
```

Analiza la siguiente tarea y propón un plan:

```
## Tarea
${input:jiraDescription:Pega la descripción de Jira}
```

```
## Lo que necesito
1. Busca en el codebase archivos relacionados.
2. Explica cómo funciona actualmente.
3. Propón plan paso a paso.
4. Identifica riesgos y dependencias.
5. Estima complejidad (baja / media / alta).
```

```
## Formato
- Pasos concretos con archivos afectados.
- Al final: tests que crear o modificar.
```

Frontmatter YAML

agent:
qué modo usar

model:
qué LLM

tools: qué skills

`${input:...}` es la clave

Cuando invocas /analyze-jira, Copilot te PIDE que pegues la descripción del Jira. No tienes que recordar el formato.

Otros: /debug-from-logs
/generate-tests /review-code

Ejercicio 1 — Crea tus archivos base

.NET

1. Crea `.github/copilotignore` para tu proyecto
2. Crea `.github/instructions/dotnet.instructions.md`
3. Crea `.github/prompts/analyze-jira.prompt.md`
4. Prueba `/analyze-jira` con una tarea real de Jira
5. Crea `.github/prompts/generate-tests.prompt.md`
6. Prueba `/generate-tests` sobre un service real

COBOL

1. Crea `.github/copilotignore` para tu proyecto
2. Crea `.github/instructions/cobol.instructions.md`
3. Crea `.github/prompts/debug-from-logs.prompt.md`
4. Prueba `/debug-from-logs` con un error real de Elastic
5. Crea `.github/prompts/analyze-jira.prompt.md`
6. Prueba `/analyze-jira` con un evolutivo de Jira

03

Custom Agents

De prompt files a compañeros de equipo especializados

Prompt file vs Custom Agent

Un prompt file es una receta. Un agente es un compañero de equipo.

	Prompt File	Custom Agent
Se invoca con	/nombre	@nombre
Tiene rol	No (solo instrucciones)	Sí (personalidad, expertise)
Persiste	No (ejecución única)	Sí (durante la sesión)
Ideal para	Tareas repetitivas concretas	Roles especializados recurrentes

¿Por qué importa la persistencia?

Si le dices "@reviewer revisa este PR", y luego le preguntas "¿y qué opinas de los tests?", **sigue actuando como reviewer**. Un prompt file habría perdido ese contexto.



Agente: @analyst

 .github/agents/analyst.md – Análisis funcional y de impacto

```
---
name: analyst
description: >
  Analista funcional que entiende el código,
  identifica impacto, propone planes.
tools:
  - codebase
  - fetch
---
```

Rol

Eres un analista funcional senior.
Entiendes .NET, COBOL y procesos de negocio.

Tu trabajo

- Analiza impacto ANTES de proponer cambios.
- Busca con codebase: ¿dónde se toca hoy?
- Identifica archivos, dependencias, copybooks.
- Propón plan paso a paso.
- Estima complejidad y riesgos.

Formato de análisis

1. Situación actual
2. Impacto del cambio
3. Plan de implementación
4. Tests necesarios
5. Riesgos

Cuándo usarlo

"@analyst analiza el impacto de añadir campo fecha_vencimiento a la tabla de pólizas"

Diferencia con /analyze-jira

El prompt file analiza UNA tarea.
El agente mantiene el rol durante toda la conversación. Le puedes pedir más contexto, aclaraciones, alternativas... sin perder el hilo.

Más agentes: @reviewer y @cobol-expert

@reviewer

 .github/agents/reviewer.md

Skills: codebase + terminal

Qué hace:

1. Ejecuta dotnet build
2. Ejecuta dotnet test
3. Revisa contra copilot-instructions
4. Checklist: seguridad > correctness > rendimiento > estilo
5. Veredicto:  /  /  + hallazgos

@cobol-expert

 .github/agents/cobol-expert.md

Skills: codebase + editFiles

Qué hace:

1. Respeta columnas 7-72 SIEMPRE
2. Incluye copybooks con COPY
3. Analiza cadena de PERFORMs
4. Avisa de copybooks compartidos
5. NO inventa DD names

Sub-agentes con /task (CLI): Un agente puede lanzar otro como subtarea en contexto aislado.

`/task "Analiza impacto de cambiar WS-IMPORTE en COPY-FACTURA"` → trabaja en paralelo, devuelve resultado.

AGENTS.md — El estándar cruzado

Un archivo en la raíz del repo, compatible con TODAS las herramientas IA

```
# AGENTS.md (raíz del repo)
# Compatible con:
# GitHub Copilot · Claude Code · Cursor
# Gemini CLI · Windsurf

## Contexto del proyecto
Solución .NET 8 multi-proyecto.
Integración con COBOL legacy z/OS.

## Convenciones
- C# 12, Clean Architecture, CQRS
- COBOL Enterprise, columnas 7-72
- xUnit + FluentAssertions
- Responder en español

## Estructura
- src/Api, Domain, Infrastructure, Cobol
- tests/

## Seguridad
- No hardcodear secrets
- Validar inputs del usuario
```

GitHub Copilot



Claude Code



Cursor



Gemini CLI



Windsurf



Si alguien del equipo usa otra herramienta IA, este archivo garantiza consistencia.

Ejercicio 2 — Crea tu agente personalizado

.NET

1. Crea `.github/agents/reviewer.md` (usa plantilla)
2. Prueba: `@reviewer` revisa el archivo que tengas abierto
3. Pídele que ejecute `dotnet build` y `dotnet test`
4. Itera: pídele correcciones sin perder el contexto

COBOL

1. Crea `.github/agents/cobol-expert.md` (usa plantilla)
2. Prueba: `@cobol-expert` explícame este programa
3. Pide análisis de impacto de un cambio en un copybook
4. Compara: ¿mejor que un prompt manual?

04

MCP Servers

Conectando Copilot con herramientas externas

¿Qué es MCP?

Model Context Protocol — el USB universal para la IA

 **Copilot**

IDE / CLI

GitHub

Atlassian / Jira

Playwright

Filesystem

MCP Protocol

¿Qué MCPs os interesan? **GitHub** (issues, PRs) · **Atlassian/Jira** (tickets, sin copiar/pegar) · **Playwright** (test web, sin credenciales)

MCP en los 3 entornos

Mismo protocolo, distinta configuración

```
// .vscode/mcp.json (VS Code)
{
  "servers": {
    "atlassian-mcp-server": {
      "type": "http",
      "url": "https://mcp.atlassian.com/v1/mcp"
    },
    "playwright": {
      "command": "npx",
      "args": ["-y",
        "@anthropic/mcp-server-playwright"]
    },
    "github": {
      "command": "npx",
      "args": ["-y",
        "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_PERSONAL_ACCESS_TOKEN":
        "${env:GITHUB_TOKEN}" }
    }
  }
}
```

VS Code → **.vscode/mcp.json**

```
// Visual Studio 2022 → .vs/mcp.json
{
  "servers": {
    "atlassian-mcp-server": {
      "type": "http",
      "url":
        "https://mcp.atlassian.com/v1/mcp"
    },
    "playwright": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y",
        "@anthropic/mcp-server-playwright"]
    }
  },
  "inputs": []
}
```

Visual Studio 2022 → **.vs/mcp.json**

Tools > Options > GitHub Copilot > MCP · Requiere v17.14+

```
# CLI
copilot mcp list
copilot mcp add-github

# Config: (Windows)
# %APPDATA%/github-copilot/mcp.json
# (Mac/Linux)
# ~/.config/github-copilot/mcp.json
```

Demo: MCP en acción

Playwright (sin credenciales) + Atlassian Jira (OAuth en navegador)

Demo 1: Playwright (todos pueden probarlo)

1. Añadir playwright a .vscode/mcp.json o .vs/mcp.json (plantilla)
2. En el chat: "Usando playwright, navega a <https://httpbin.org/get> y dime qué headers envía el navegador"
3. Copilot abre un navegador headless, navega, y te devuelve la respuesta

Demo 2: GitHub MCP

"Lista los últimos 5 issues abiertos del repo [nombre]"

Requiere: GITHUB_TOKEN

Demo 3: Atlassian / Jira MCP

"Lee el ticket JIRA-1234 y propón implementación usando @workspace para encontrar código"

OAuth: se abre navegador al conectar

🕒 Ejercicio: configurad Playwright MCP y probadlo · Si tenéis Jira, probad Atlassian (5-10 min)

05

Copilot Memory

Contexto que persiste entre sesiones



¿Qué es Copilot Memory?

SIN Memory

Sesión 1: "Usamos C# 12 con EF Core"

→ Copilot responde bien

Sesión 2: "Añade un endpoint"

→ No recuerda nada de C# 12

Sesión 3: "Usamos C# 12..." (otra vez)

CON Memory

Sesión 1: "Usamos C# 12 con EF Core"

→ Copilot lo guarda

Sesión 2: "Añade un endpoint"

→ Ya sabe que usáis C# 12 y EF Core

Sesión 3: sigue recordando

Memory vs Instructions

Necesidad	Memory	Instructions	Por qué
"Siempre responde en español"	Funciona	✓ Mejor	Aplica a todo el equipo
"Prefiero código conciso"	✓ Perfecto	—	Es personal, no del repo
"Usamos xUnit en este repo"	—	✓ Mejor	Es del repo, no personal
"Trabajo en módulo facturas"	✓ Perfecto	—	Temporal, entre sesiones

Regla: del equipo/repo → instructions. Personal → memory. Temporal → memory.

06

Workflows

Todo junto: el flujo diario con Copilot configurado

El workflow diario con todo configurado

1 /analyze-jira

Prompt file

Prompt file analiza la tarea.
codebase busca archivos.



2 @analyst

Agent

Agente valida el plan contra
arquitectura y convenciones.



3 Agent mode

Modo

Implementa siguiendo el plan.
editFiles + terminal.

4 /generate-tests

Prompt file

Genera tests y ejecuta
dotnet test.



5 @reviewer

Agent

Revisa cambio completo.
Build + test + checklist.



6 git commit

Auto

Copilot sugiere mensaje
basado en los cambios.

Sin esta configuración, cada paso es un prompt manual desde cero. Con ella, es un flujo estandarizado.

Cómo fluye el contexto en este workflow

`copilot-instructions.md`

SIEMPRE activo

`dotnet.instructions.md`

Activo con archivos .cs

`tests.instructions.md`

Activo con archivos de test

`/analyze-jira`

Puntual (ejecución única)

`@analyst`

Persiste en la conversación

`Memory`

Recuerda entre sesiones

`MCP (Jira, GitHub)`

Lee datos externos en vivo

Cada capa añade contexto SIN que tengáis que escribirlo. Esa es la clave.

Checklist: repo listo para Copilot

☒	<code>.github/copilot-instructions.md</code>	Contexto global del proyecto
☒	<code>.github/instructions/*.instructions.md</code>	Reglas por lenguaje (.NET, COBOL, tests)
☒	<code>.github/prompts/*.prompt.md</code>	Tareas repetitivas automatizadas
☒	<code>.github/agents/*.md</code>	Roles especializados del equipo
☒	<code>.github/copilotignore</code>	Exclusiones de seguridad y contexto
☒	<code>.github/skills/*/SKILL.md</code>	Skills: dotnet-review, cobol-analyzer
☒	<code>.vscode/mcp.json + .vs/mcp.json</code>	MCP: Atlassian, Playwright, GitHub
☒	<code>AGENTS.md</code>	Compatibilidad cross-tool
☐	Memory configurada	Preferencias personales

Todo está en la plantilla ZIP. Copiad, adaptad, committed.

Ejercicio final — Workflow completo con tarea real

.NET

1. /analyze-jira → analiza tu tarea real
2. @reviewer → valida el plan
3. Agent mode → implementa
4. /generate-tests → crea tests
5. @reviewer → revisión final

¿El flujo fue más ágil que antes?

COBOL

1. /analyze-jira o /debug-from-logs
2. @cobol-expert → análisis impacto
3. Agent mode → implementa el cambio
4. @cobol-expert → revisa convenciones

¿Copilot tuvo mejor contexto?

Lo que habéis construido hoy



.copilotignore — seguridad y contexto limpio



Instructions path-specific (.NET, COBOL, tests)



Skills con SKILL.md (dotnet-review, cobol-analyzer)



5 prompt files para tareas diarias



3 custom agents (analyst, reviewer, cobol-expert)



MCP configurado (Atlassian, Playwright, GitHub)



Entendéis Memory y cómo complementa instructions



AGENTS.md para compatibilidad cruzada

Podéis commitar el .github/ a vuestro repo HOY. Todo el equipo se beneficia.

Tarea para la semana

1

Commitea tu .github/ al repo de tu equipo

2

Pide a un compañero que pruebe tus prompt files y agentes

3

Anota qué funciona y qué no — lo revisamos en la próxima sesión

4

Si te atreves: crea un agente nuevo para una tarea que repitas cada semana

5

Configura el MCP de Atlassian/Jira (pide token a tu admin)

07

Quiz

¿Cuánto habéis aprendido hoy? Vamos a comprobarlo

Pregunta 1/6

¿Dónde se guarda un custom skill de Copilot en el repositorio?

A

`.github/prompts/nombre.prompt.md`

B

`.github/skills/nombre/SKILL.md`

C

`.github/agents/nombre.md`

D

`.github/copilot-instructions.md`



Respuesta: B Los skills van en `.github/skills/nombre/` con un `SKILL.md` obligatorio.

Pregunta 2/6

¿Cuál es la diferencia principal entre un Tool y un Skill?

A

Son lo mismo, sinónimos

B

Los tools los crea el usuario, los skills vienen built-in

C

Los tools son capacidades built-in; los skills son paquetes de conocimiento que tú creas

D

Los tools solo funcionan en CLI



Respuesta: C Tools = brazos (built-in). Skills = libros de recetas (los creas tú).

Pregunta 3/6

¿Qué archivo de MCP usas en Visual Studio 2022?

A

`.vscode/mcp.json`

B

`.vs/mcp.json`

C

`~/.copilot/mcp.json`

D

`copilot-instructions.md`



Respuesta: B VS 2022 usa `.vs/mcp.json`. VS Code usa `.vscode/mcp.json`.

Pregunta 4/6

¿Qué tipo de archivo se carga SIEMPRE en cada interacción con Copilot?

A

Prompt files (.prompt.md)

B

Skills (SKILL.md)

C

Custom Agents (.agent.md)

D

Instructions (copilot-instructions.md)



Respuesta: D Las instructions se inyectan en CADA request. El resto se carga bajo demanda.

Pregunta 5/6

¿Qué ventaja tiene un Skill sobre un Prompt File?

A

Los skills son más rápidos

B

Auto-descubrimiento por el agente + recursos adjuntos + progressive loading

C

Los skills funcionan en más entornos

D

Los prompt files están obsoletos



Respuesta: B Los skills se descubren solos, cargan progresivamente, y pueden incluir scripts, docs, y código.

Pregunta 6/6

¿Qué hace Copilot Memory?

A

Guarda el código del proyecto entre sesiones

B

Recuerda preferencias y contexto entre sesiones de chat

C

Reemplaza las instructions del repositorio

D

Almacena los resultados de los tests



Respuesta: B Memory recuerda preferencias personales entre sesiones. No reemplaza instructions.



*El mejor Copilot no es el que tiene
el modelo más potente. Es el que tiene
el contexto más preciso.*



¿Preguntas?

¡Gracias!